

Contents

SPECIFICATION — RAG Eval Harness (BM25 + LLM-as-judge + uv)	1
0. Intent and purpose	2
1. Actors and goals	3
2. Requirements (intent, high level)	3
3. Behavior and state model	5
3.1 Per-case pipeline state machine (CaseState , one question at a time)	5
3.2 Threading / I/O model	6
3.3 The retrieval → context → generation → judge data flow	6
4. Interfaces / contracts	6
C-01 Corpus and documents	6
C-02 Retrieval (BM25, deterministic — R-02, R-17)	6
C-03 Context construction (token budget — R-03, §14, §3)	7
C-04 Question / Answer / Verdict record types	7
C-05 LLM interface (the one probabilistic component — R-05, R-17)	8
C-06 Judge interface (LLM-as-judge — R-06, §19/§20)	9
C-07 Metrics (metrics.py , pure — R-07/08/09/12)	9
5. Interface specification	10
5.1 CLI — primary surface (rag-eval , R-11)	10
5.2 GUI — optional surface (rag-gui , R-13, ch1 style)	11
6. Invariants (must hold in every valid implementation)	11
7. Constraints (precise and measurable)	12
8. Edge cases and failure semantics	13
9. Acceptance criteria, tests, and evals	14
9.1 Corpus and question dataset (§15, §17)	14
9.2 Retrieval (deterministic — R-02, R-17, no LLM)	15
9.3 Context + budgets (deterministic — R-03, §14, §3)	15
9.4 LLM + Judge + Metrics (offline, MockLLM + MockJudge — R-05/06/07/08/09)	15
9.5 Manual / real-model smoke (opt-in — not in uv run pytest)	16
9.6 Structure / architecture (ch1 T-02 analog — I-009, R-17)	16
10. Dependencies and environment	16
11. Traceability matrix (id → where realized)	18

SPECIFICATION — RAG Eval Harness (BM25 + LLM-as-judge + uv)

- **Status:** v0.1 — draft for implementation
- **Language:** Python 3.12 | Retrieval: pure-Python BM25 | GUI: PyQt5 (optional) | HTTP: httpx | Schema: jsonschema | LLM: local Ollama
- **Curriculum source:** **curriculum/week1/chapter2.md** (§15 An Experimental System, §16 Start With Easy Questions, §17 Increase the Difficulty, §18 Retrieval Metrics, §19 Answer Accuracy, §20 Hallucination Rate, §21 The Context Engineering Creates an Eval Loop).
- **Scope of this document:** the *authoritative specification* of an AI-native system. It is written to Level 2–3 (structured, mostly executable): behavior, interfaces, invariants, edge cases, and failure semantics are made explicit so an agent (or engineer) can derive implementation **and** verification with minimal inference.
- **Principle:** requirements express *intent*; this specification *operationalizes* intent into observable behavior plus the conditions under which we know it is correct.

0. Intent and purpose

Chapter 2’s central lesson is that application quality is a property of **the context presented to the model**, not of the prompt:

LLM Application = State + Retrieval + Context Construction + Model + Tools + Evaluation

This lab is the *experimental system* of §15–§21: a small but **measurable** retrieval-augmented generation pipeline plus the **eval loop** that makes it an engineering artifact rather than a demo. The user asks a question; the system (1) **retrieves** relevant documents from a corpus, (2) **constructs** the context (dedupe, rank, truncate to a token budget, label sources), (3) **grounds** an answer in that context through a local model, and (4) **evaluates** the result — retrieval precision/recall, answer accuracy, and hallucination rate — with ground truth.

The pipeline instantiates §13 (the context-engineering pipeline: *intent* → *retrieval* → *construction* → *LLM* → *structured output*) and §3 (the context is a *resource* with capacity, cost, and quality limits). Crucially it instantiates the chapter’s sharpest claim (§12, §21):

Context construction should be deterministic and testable wherever possible.

Accordingly the spec splits the system along the chapter’s reliability boundary (§15):

- **Deterministic boundary (pure, offline, reproducible, no LLM, no network):** the **BM25 retriever**, the **context builder / token budget**, the **metric math** (precision, recall, F1, hallucination rate), the **corpus + question generator** (ground truth), and the **eval harness** that wires them.
- **Probabilistic boundary (the one unreliable component):** the **local LLM** (default qwen3.8:27b-mlx, served by Ollama) used in exactly two places — *answer generation* and *LLM-as-judge* verdicting (§19 “structured evaluation: ask an evaluator to classify”). Both are isolated behind interfaces and replaced by deterministic **MockLLM / MockJudge** test doubles so the whole suite runs offline (ch1 §9 philosophy, carried forward).

Deployment decision (ch1 §0, “APIs vs. Local Models”): generation and judging are *local*, performed by the **Ollama** runtime (default `http://localhost:11434`) against the **already-pulled** model qwen3.8:27b-mlx (ollama list ID 5642e97495e1, ~18 GB). The app talks to Ollama’s HTTP `/api/chat`. When Ollama or the model is unavailable the system **degrades to the MockLLM/MockJudge doubles** and says so in the UI/CLI banner (§§ E-13, E-14) — it never requires a network to import, build, or test.

Primary product surface: a **CLI eval harness (rag-eval)** that runs the pipeline over a *question dataset with known answers and known supporting documents* (§15) and prints a **metrics report** (human + machine-readable JSON) plus a per-tier breakdown — this is “eval-driven development” (§21). An **optional PyQt5 GUI (rag-gui, mirroring ch1)** lets the user type one question, watch the ranked retrieval + BM25 scores, see the grounded answer with cited sources, and read the judge verdict {`correct`, `supported`, `complete`} for that query — reusing the *same* pipeline modules.

Non-goals (explicit, to constrain the solution space):

- **No learned/dense embeddings.** Retrieval is **deterministic lexical (BM25)** — no `nomic-embed-text`, no vector DB, no second model. This is a deliberate design decision from §12/§15 (“deterministic and testable wherever possible”) and keeps the eval reproducible. Dense-vector retrieval is an *out-of-scope* extension (Q-01).
- **No conversation / multi-turn.** History is a *context* input (§2), not a feature to build; each question is a single, independent inference.
- **No tool-calling loop.** §7 tool context is conceptual; the “tools” here are the *deterministic pipeline stages* (retrieve → construct → generate → judge), not a model-driven function-call loop.
- **No state/memory subsystem** (§10 State, §11 Memory): the corpus + question set are static per run; no persistence between questions. (Noted as an extension, Q-06.)
- **No vector/embedding index, no chunking of large web content.** The corpus is a folder of ~100 short, self-contained `.txt` documents (§15) — not a chunked web crawl.

- The app must **not require Ollama** to import, build, or run its **test suite**: a deterministic **MockLLM** + **MockJudge** provide every capability for offline/CI use. Ollama + **qwen3.8:27b-mlx** are the *real* generation/judge backend for the manual smoke eval.

1. Actors and goals

Actor	Goals
User (human, single process)	Run the eval (CLI) over the question dataset; inspect a metrics report and per-tier breakdown; and/or, in the optional GUI, type one question and see its retrieval ranking, grounded answer, cited sources, and verdict.
BM25 Retriever (<code>retrieval.py</code>)	Given a query, return a ranked ScoreDoc list over the corpus. Deterministic , pure-Python, no LLM/network.
Context Builder (<code>context.py</code>)	Turn the ranked docs into a token-bounded, deduped, source-labeled Context (the text the LLM actually sees). Deterministic, no LLM/network.
LLM (<code>model.py</code> : OllamaLLM real, MockLLM offline double)	Given system + context + question, produce a grounded, structured answer {answer, confidence, sources} . Never touches the CLI/GUI.
Judge (<code>judgment.py</code> : OllamaJudge real, MockJudge offline double)	Given question + context + generated answer + gold answer + relevant docs, classify {correct, supported, complete} and enumerate unsupported claims (§19/§20). Never touches CLI/GUI.
Ollama daemon (<i>external</i>)	Local inference runtime at <code>http://localhost:11434 (/api/chat, /api/tags)</code> . Model qwen3.8:27b-mlx for both generation and judging. Owns the weights + CPU/GPU/NPU. Not part of this project; E-13 / E-14 handle its absence.
Corpus / Generator (<code>corpus.py</code> , <code>gen_corpus.py</code>)	Load the 100-doc corpus from <code>documents/</code> and generate the ground-truth question dataset (question + relevant_documents + answer , §15). Deterministic (seeded), so the eval has a stable baseline.
Eval Harness (<code>eval.py</code> , <code>cli.py</code>)	Wire the stages per question, accumulate RunMetrics , and aggregate to the dataset-level report (§§18–21).
Metrics (<code>metrics.py</code>)	Compute precision/recall/F1 (from TP/FP/FN, §18), answer accuracy, and hallucination rate (§20) — pure, headless, testable.
UI (<code>ui.py</code> , <i>optional</i>)	One-question interactive view over the shared pipeline; never blocks on inference.

2. Requirements (intent, high level)

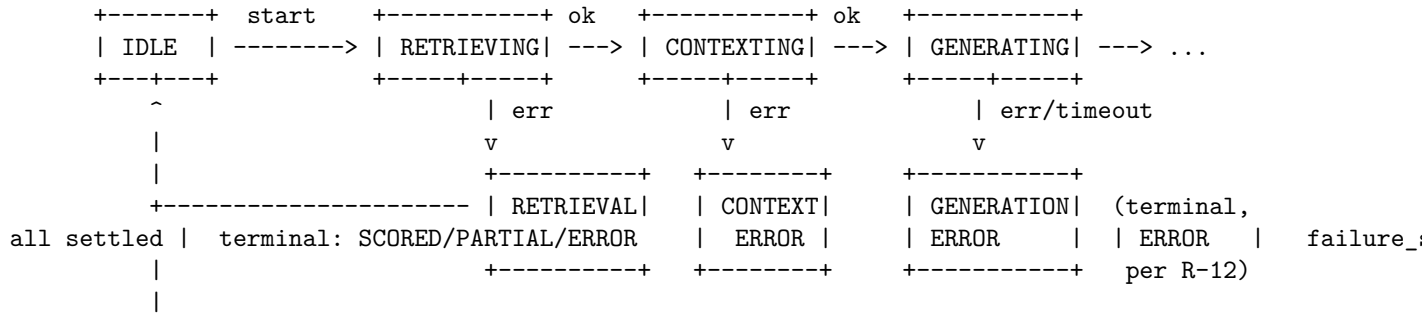
ID	Statement
R-01	The system shall build a retrieval pipeline question → retrieval → context construction → LLM → structured answer → judge → metrics over a corpus of ≈ 100 short documents (§15).
R-02	The retriever shall be deterministic lexical BM25 (pure-Python, no embedding model, no network): given the same corpus + query + parameters it returns a byte-identical ranked ScoredDoc list (§12; non-goal).
R-03	The context builder shall turn the ranked docs into a single Context with an enforced token budget ($B_{\text{retrieval}} \leq B_{\text{total}}$, §14): dedupe, rank by score, truncate by dropping lowest-score docs first when over budget, and label each included source .
R-04	Every document the answer cites (its sources) and every claim the judge marks <i>supported</i> shall be traceable to a doc id present in the retrieved context — no fabricated citations or grounds (anti-hallucination gate, §20/§21).
R-05	The pipeline shall emit a grounded, structured answer { answer , confidence , sources } via the local LLM, validated the ch1 way: raw text → strip optional ``json fence → json.loads → schema-validate → accept/ reject-with-retry (§15, ch1 C-05).
R-06	The judge shall classify each answer LLM-as-judge (§19) into a structured verdict { correct , supported , complete , unsupported_claims , total_factual_claims , rationale } produced by the local LLM and schema-validated; offline, a deterministic MockJudge supplies verdicts.
R-07	The system shall measure retrieval precision and recall with the §18 TP/FP/FN decomposition: $\text{Precision} = \text{TP}/(\text{TP}+\text{FP})$, $\text{Recall} = \text{TP}/(\text{TP}+\text{FN})$, plus F1, using the question’s relevant_documents as ground truth (§18).
R-08	The system shall measure answer accuracy from the verdict’s correct field (§19), aggregated over judged rows.
R-09	The system shall measure the hallucination rate as $\text{unsupported_claims} / \text{total_fational_claims}$ (§20), aggregated over judged rows, with a defined no-division-by-zero behavior when total claims is 0.
R-10	The question dataset shall contain known answers and known supporting documents (§15), organized into the §17 difficulty tiers : easy (1 doc), multi (≥ 2 docs), synthesis (combined facts), distractor (lexically similar but irrelevant / contradictory docs).
R-11	The primary product surface is a CLI eval harness (rag-eval) that runs the full pipeline over the dataset and emits a metrics report — machine-readable JSON and a human-readable summary — including a per-tier breakdown (§21).
R-12	The system shall attribute every failed case to a specific stage (retrieval context generation judging) so the report distinguishes “ <i>did retrieval fail to provide evidence</i> ” from “ <i>did the model fail to use it</i> ” (§18, the central diagnostic of the chapter).
R-13	An optional PyQt5 GUI (rag-gui) reusing the <i>same</i> retrieval/context/model/judgment/metrics modules shall let the user type one question and inspect ranked retrieval (with BM25 scores), the grounded answer with cited sources, and the verdict — one question at a time.
R-14	The project shall be reproducible via uv on Python 3.12 and fully offline (no Ollama, no network) for the entire automated test suite via MockLLM + MockJudge ; the real qwen3.8:27b-mlx path is an opt-in / manual smoke (§10, ch1 R-14).
R-15	With a fixed seed on the mock path, all deterministic outputs (retrieved ids, context, computed metrics, corpus/question generation, mock answers/verdicts) are reproducible ; the real Ollama path is best-effort reproducible (token counts/metrics asserted, exact generated text is not — ch1 R-15).
R-16	On start the real path shall discover locally-pulled Ollama models via GET /api/tags ; if Ollama is unreachable it shall fall back to MockLLM/MockJudge and state so (CLI banner / GUI banner). If --model names a model that is not pulled locally , that model’s stage surfaces a clear “pull required” error rather than a crash (ch1 R-16/R-17 , E-13/E-14).

ID	Statement
R-17	The LLM is used in exactly two places — answer generation (§5 answer schema) and judging (§6 verdict schema). Retrieval, context construction, and metric computation never call the LLM (the deterministic boundary, §12). Asserted by an import/structure scan (T-02).

3. Behavior and state model

3.1 Per-case pipeline state machine (CaseState, one question at a time)

Each question runs through the §13 pipeline as a linear, single-threaded sequence of stages with a terminal outcome. A failure at any stage terminates **that case** but leaves other cases in the suite running (one question never poisons the next).



State	Meaning	Terminal?
IDLE	Case scheduled; not yet started.	no
RETRIEVING	BM25 <code>search(query, k)</code> running over the corpus.	no
CONTEXTING	<code>build_context(scored, token_budget)</code> — dedupe/rank/truncate/label.	no
GENERATING	<code>LLM.generate(system, context, question)</code> → structured answer (may retry parse).	no
JUDGING	<code>Judge.judge(...)</code> → verdict (may retry parse).	no
SCORED	All stages ok: Answer + Verdict + RunMetrics (incl. retrieval metrics) complete.	yes
PARTIAL	Retrieval + generation ok but judge failed (E-15): retrieval metrics recorded; <code>correct/supported/complete = None</code> , <code>failure_stage="judging"</code> . Row still counts for retrieval metrics.	yes
ERROR	A stage before judging terminal-faulted: <code>failure_stage</code> names the failing stage (retrieval, context, or generation).	yes

Transition rules:

- Stages are **strictly ordered**: a case that reaches **GENERATING** has a successful **CONTEXTING**; a case in **PARTIAL/SCORED** has a successful **RETRIEVING** and **GENERATING**. Retrieval-stage output (`retrieved`, `expected`, `tp/fp/fn`, `precision`, `recall`, `f1`) is populated for **every** case that cleared **RETRIEVING**, regardless of later outcome — so a generation or judge failure still yields a **complete retrieval diagnosis** (§18 “did retrieval fail?”).
- A stage’s parse/validation failure triggers **retry up to max_retries** with an **error-informed** prompt (the prior attempt’s failure appended as a directive), then terminal failure for that stage with the **first** failure reason recorded — mirroring ch1 E-03; the deterministic boundary rejects, never fabricates.
- The whole suite is settled when **every** case is terminal (**SCORED/PARTIAL/ERROR**). `--stop-on-error` (opt-in) aborts on first non-terminal fault; **default is run-all**.

3.2 Threading / I/O model

- **CLI (rag-eval):** a **sequential** per-case loop. Each case is independent; the loop never blocks the terminal on a single model call longer than one `timeout_s`. With the slow `qwen3.8:27b-mlx` a case can take seconds; the CLI prints per-case progress to `stderr` so the operator sees advancement. **No default parallelism** (a single shared `qwen3.8:27b-mlx` Ollama instance is one inference slot; a `--concurrency N` opt-in is *out-of-scope* for v0.1, Q-05 — keep it deterministic and simple).
- **GUI (rag-gui, optional):** the pipeline runs **off the Qt event-loop thread** in a `QThread` worker (ch1 §3.3 pattern); the UI only updates from queued signals. A `Cancel` tears the worker down and surfaces a terminal `ERROR` panel (E-16).

3.3 The retrieval → context → generation → judge data flow

```
question.question
|
BM25.search(k) -----> [ scored: ScoredDoc[] ]      (deterministic, §12)
|
build_context(budget,k') -----> Context{docs, prompt, provenance, tokens, truncated}
|                               | (deterministic, token-bounded §14)
LLM.generate(system, ctx.prompt, q) -----> Answer{answer, confidence, sources[]}
|                               | (PROBABILISTIC - the one unreliable step)
Judge.judge(q, ctx, answer, gold, relevant) -> Verdict{correct,supported,complete,...}
|                               | (PROBABILISTIC - LLM-as-judge)
metrics.compute(ctx, retrieved, expected, verdict) -> RunMetrics
```

The **deterministic** stages (retrieve, context, metrics) and the **probabilistic** stages (generate, judge) are separated by the ch1 §15 reliability boundary. The LLM appears **only** in generate + judge (R-17; enforced by T-02 import scan).

4. Interfaces / contracts

C-01 Corpus and documents

```
@dataclass
class Document:
    doc_id: str          # stable id, e.g. "policy-17", "travel-03", "001" (must equal
                        # the §15 filename stem when loaded from documents/)
    text: str            # full document text
    domain: str | None = None # coarse category used for distractor grouping + provenance labels

def load_corpus(path: str) -> list[Document]:
    """Load `documents/NNN.txt` (or a single .jsonl) into Document[]. doc_id =
    filename stem by default. Raises on a malformed entry (E-01). Pure, no LLM/network."""

def generate_corpus_and_questions(out_dir: str, n_docs: int = 100, n_questions: int = 25,
    seed: int = 42) -> None:
    """Deterministically write the 100-doc corpus under out_dir/documents/ AND a
    grounded `questions.json` (§15) with the §17 tiers. Seeded => reproducible (R-15).
    Ground truth (question<->relevant_documents) is fixed at generation time."""
```

C-02 Retrieval (BM25, deterministic — R-02, R-17)

```
class BM25Retriever:
    def __init__(self, documents: list[Document], *, k1: float = 1.5, b: float = 0.75,
        stop_words: frozenset[str] | None = None) -> None: ...
```

```
def search(self, query: str, k: int) -> list[ScoredDoc]:
    """Return up to k documents ranked by BM25 desc. Deterministic: same
    corpus + query + (k1,b,tokenize) => byte-identical result list (R-02).
    Ties broken by doc_id ascending then score desc (stable, documented). Empty
    corpus or no matches => [] """
```

@dataclass ScoredDoc: doc: Document, score: float >= 0.0, rank: int >= 1.

BM25 (O-1 reference, O-1a tokenizer, O-1b tie-break are the exact formulas the T-suite pins):

$$\text{score}(q, d) = \sum_{t \in q} \text{idf}(t) \cdot \frac{\text{tf}(t, d) (k_1 + 1)}{\text{tf}(t, d) + k_1 (1 - b + b \frac{|d|}{\text{avgdl}})} \quad \text{idf}(t) = \ln \left(1 + \frac{N - n(t) + 0.5}{n(t) + 0.5} \right)$$

Tokenizer (O-1a): lowercase; split on `[^\w']+`; drop pure-numeric tokens unless `--keep-numbers`; apply optional stop-word filter; **deterministic dict order** (no hash-seed dependence). **Ties (O-1b):** equal scores resolve `doc_id` ascending, so runs are byte-reproducible on a given corpus (R-15). No LLM/network (R-17).

C-03 Context construction (token budget — R-03, §14, §3)

```
@dataclass
class Context:
    docs: list[ScoredDoc]          # included, ranked, deduped
    prompt: str                    # the assembled text the LLM sees (with source labels)
    provenance: list[str]          # doc_ids included, in include-order (== docs order)
    tokens: int                    # est_tokens(prompt) <= token_budget (I-006)
    truncated: bool                # True iff one or more docs were dropped to fit the budget

def build_context(scored: list[ScoredDoc], *, token_budget: int, dedupe: bool = True) -> Context:
    """Dedupe (by normalized text; keep highest-score), rank (already by score desc),
    then include docs from highest score down until est_tokens(prompt) would exceed
    token_budget: drop the lowest-score remaining docs first; set truncated=True iff any
    dropped. Each included doc's text is emitted in the prompt with a `[doc_id]` source
    label. Pure, deterministic, no LLM/network (R-17)."""
```

Token estimation (O-2): `est_tokens(s)` is a deterministic, documented heuristic (default `len(text.split()) * 5 + 4` words is ~5 chars... no — define precisely: `ceil(len(s) / 4)`; the same formula must be used by the builder and by every report, so the budget math in the report matches what was actually built — I-006). It is *explicitly an estimate*, not a model tokenizer (§3: capacity is a resource; exact tokenizer-accurate counts are out of scope, Q-04).

C-04 Question / Answer / Verdict record types

```
@dataclass
class Question:
    q_id: str
    question: str
    gold_answer: str
    relevant_docs: list[str]      # §15/§18 ground truths (the TP/FN universe for this q)
    tier: str                     # one of {"easy", "multi", "synthesis", "distractor"}

# §19 answer schema (produced by the LLM, schema-validated like ch1 C-05):
# { "answer": str(minLength 1), "confidence": number[0,1], "sources": array of doc-id strings }

@dataclass
class Answer:
    q_id: str
```

```

text: str          # the free-form answer (Answer.answer)
confidence: float  # [0,1]
sources: list[str] # doc_ids the model claims to cite; must be a subset of context.provenance (I-00)
usage: "Usage"     # ch1 C-01 Usage (prompt/completion tokens, total), for observability
status: str        # "COMPLETED" | "ERROR" (parse/validation exhausted)

# §19/§20 verdict schema (produced by the JUDGE LLM, schema-validated):
# { "correct": bool, "supported": bool, "complete": bool,
#   "unsupported_claims": array of strings, "total_factual_claims": integer(min 0),
#   "rationale": string }
@dataclass
class Verdict:
    q_id: str
    correct: bool
    supported: bool          # every factual claim grounded in the retrieved context
    complete: bool          # all facts in the question's relevant_docs are reflected
    unsupported_claims: list[str]
    total_factual_claims: int # >= 0; denominator for the hallucination rate (§20)
    rationale: str
    status: str              # "JUDGED" | "ERROR" | "SKIPPED"

```

Schemas (JSON Schema, ch1 C-05 style, `additionalProperties:false`): the *answer* schema and the *verdict* schema are both defined as JSON Schema objects in the repo (e.g. `schemas/answer.json`, `schemas/verdict.json`) and validated by `jsonschema` after an optional single-````json` fence is stripped (ch1 E-11). Both share ch1's `parse→validate→accept/reject` pipeline and a `max_retries` error-informed retry.

C-05 LLM interface (the one probabilistic component — R-05, R-17)

```

class LLM(ABC):
    @property
    def model_id(self) -> str: ...          # e.g. "qwen3.8:27b-mlx" or "mock"

    @abstractmethod
    def generate(self, *, system: str, context: str, question: str, schema: dict,
                 max_tokens: int = 512, temperature: float = 0.0, seed: int | None = 42,
                 max_retries: int = 2,
                 on_failure: str | None = None) -> Answer:
        """Produce a STRUCTURED §19 answer by prompting the model to emit the answer
        schema object. Validate like ch1 C-05 (fence->json.loads->jsonschema). On
        failure, retry up to max_retries with `on_failure` (the last parse error) appended
        to the system prompt; if all exhausted, return Answer(status="ERROR") with the
        first failure reason - never an unvalidated dict. `sources` must reference ids that
        appear in `context` (the model is told to cite only provided docs; I-003 is enforced
        deterministically by the harness, not trusted to the model)."""

class OllamaLLM(LLM): # the real backend; POST /api/chat -> qwen3.8:27b-mlx (ch1 C-03b shape)
class MockLLM(LLM):   # deterministic offline double: derives a canned, schema-valid Answer
                        # from the question+context so the T-suite needs no Ollama (R-14).

```

OllamaLLM is the **only** module that names an Ollama URL/model shape (ch1 I-002; a source scan T-02 asserts this). It reuses ch1's OllamaClient transport (`httpx`, `/api/chat`, `/api/tags`, `NDJSON`, `prompt_eval_count/eval_count→Usage`).

C-06 Judge interface (LLM-as-judge — R-06, §19/§20)

```
class Judge(ABC):
    @property
    def model_id(self) -> str: ...           # may be "" (deterministic) or a model id

    @abstractmethod
    def judge(self, *, question: Question, context: Context, answer: Answer,
              max_retries: int = 2,
              on_failure: str | None = None) -> Verdict:
        """Produce a §19/§20 Verdict. `supported` is True only when every factual claim
        in answer.text is grounded in context.provenance; `unsupported_claims` lists claims
        not traceable to any provided doc; `total_factual_claims` is the denominator for the
        hallucination rate. Validates the verdict schema like ch1 C-05; on exhaustion returns
        Verdict(status="ERROR")."""

class OllamaJudge(Judge): # real: asks qwen3.8:27b-mlx to emit the verdict schema (R-06)
class MockJudge(Judge):  # deterministic offline double: verdicts derived from ground truth
                        # (intersection of question.relevant_docs and context.provenance, and gold_a
                        # answer.text) so the T-suite asserts metric MATH without a model.
```

Design note (Q-03): the judge is a *second* LLM role but the **same** model (qwen3.8:27b-mlx) by default, per your answer. Judge.model_id may independently name a model, so a cheaper model could be swapped for judging later. The offline MockJudge reproduces verdicts **deterministically from ground truth** so the automated suite measures the *metric computation* (R-07/08/09), not a model's judgment.

C-07 Metrics (metrics.py, pure — R-07/08/09/12)

```
@dataclass
class RunMetrics:
    q_id: str
    tier: str
    # -- retrieval (populated iff case cleared RETRIEVING) --
    retrieved: list[str]          # doc_ids returned, in rank order
    expected: list[str]          # == question.relevant_docs
    tp: int; fp: int; fn: int
    precision: float | None      # TP/(TP+FP); None if TP+FP==0 (E-02)
    recall: float | None         # TP/(TP+FN); None if TP+FN==0 (E-03)
    f1: float | None             # 2PR/(P+R) when P+R>0 else 0.0 (E-04)
    context_tokens: int
    truncated: bool
    # -- answer + judge (populated if generation/judging ran) --
    answer_status: str           # "COMPLETED" | "ERROR"
    correct: bool | None
    supported: bool | None
    complete: bool | None
    unsupported_claims: int
    total_factual_claims: int
    # -- diagnostics / timing --
    failure_stage: str | None    # None | "retrieval" | "context" | "generation" | "judging" (R-12)
    retrieve_ms: float
    generate_ms: float
    total_latency_ms: float
    status: str                  # "SCORED" | "PARTIAL" | "ERROR" (§3.1)
```

```

@dataclass
class AggregateMetrics:
    n_cases: int
    # dataset-level means over relevant rows:
    precision: float          # mean of non-None per-row precision
    recall: float
    f1: float
    answer_accuracy: float    # mean(correct) over JUDGED rows (R-08)
    hallucination_rate: float # sum(unsupported_claims)/sum(total_factual_claims) over JUDGED rows (R-09)
    failure_breakdown: dict[str, int] # failure_stage -> count (R-12)
    by_tier: dict[str, "AggregateMetrics"] # per §17 tier (R-11)

def retrieval_pr(expected: list[str], retrieved: list[str]) -> tuple[int, int, int, float, float, float]:
    """(tp,fp,fn,precision,recall,f1) with the I-001/007 guards: empty denominators =>
    the metric None and a 0.0 f1; non-None metrics use the §18 worked-example arithmetic."""

def aggregate(rows: list[RunMetrics]) -> AggregateMetrics:
    """Mean precision/recall/f1 over rows with non-None values; hallucination rate with the
    I-007 no-division-by-zero guard; per-tier recursion; failure_breakdown from failure_stage."""

§18 worked example (the T-suite pins this): expected = {D3, D17, D42}, retrieved = [D3, D17,
D88, D91] ⇒ TP = 2, FP = 2, FN = 1, Precision = 2/4 = 0.50, Recall = 2/3 ≈ 0.667, F1 =  $\frac{2PR}{P+R} \approx 0.571$ .
This exact tuple is asserted by T-05.

```

5. Interface specification

5.1 CLI — primary surface (rag-eval, R-11)

Usage: rag-eval <command> [options]

Commands:

eval	Run the full pipeline over a question dataset and emit a metrics report.
gen-corpus	Generate the ~100-doc corpus + grounded questions.json (deterministic, §15).
show	Print one case's retrieval+context+answer+verdict (diagnostic / "what world did the model

Common options:

--dataset PATH	questions.json (default: ./questions.json)
--corpus PATH	document directory or .jsonl (default: ./documents)
--out PATH	write the JSON report (default: report.json; also prints to stdout -h)
--k N	retrieval top-k (default 5)
--budget N	retrieval token budget B_retrieval (default 2000)
--tiers LIST	subset of {easy,multi,synthesis,distractor} (default: all, §17)
--model NAME	Ollama model for generate+judge (default qwen3.8:27b-mlx)
--judge on off	run LLM-as-judge (default on); off = retrieval-only eval (R-12 diagnostic)
--mock	force the offline MockLLM+MockJudge path (no Ollama, no network)
--seed N	determinism seed (default 42) (R-15)
--stop-on-error	abort after the first non-terminal fault (default: run all)
--quiet	suppress per-case stderr progress

eval output:

1. A **human-readable summary** to stdout: aggregate precision / recall / f1 / answer_accuracy / hallucination_rate, the per-tier breakdown, and the failure_breakdown (so the operator im-

mediately sees whether failures cluster in retrieval vs generation vs judging — §18/§21’s point).

2. A **machine-readable JSON report** to `--out` (and, `--quiet-less`, `stdout`) with one `RunMetrics` record per case plus the `AggregateMetrics` — the artifact that makes a change a *measurable* result (“raising k from 5→8 moved precision 0.71→0.84, recall 0.93→0.89, accuracy +4.2%”, §21).

Exit codes: 0 = ran (even if some cases errored — errors are *recorded*, per §3.1 run-all default); 2 = bad usage/CLI args; 3 = corpus/questions load failure (E-01/E-17); 4 = fatal backend failure with `--mock` unavailable (E-13 hard error *only* when not auto-falling-back). A per-case non-terminal fault does **not** set a failing exit code (it is a *result* the report carries, R-12).

5.2 GUI — optional surface (rag-gui, R-13, ch1 style)

Reuses the **same** retrieval/context/model/judgment/metrics modules; the only new code is a `QThread` worker + widgets. Layout:

```
+-----+
| RAG Eval Harness - context = the program (ch2 §1) |
+-----+
| QUESTION (QPlainTextEdit) | RETRIEVAL (ranked) | ANSWER + VERDICT | | | |
| model / k / budget / tiers spin| doc_id score | ctx | text | confidence | [Run] [Cancel] |
| [Run] state label | [001] 3.42 | Y | ... | sources: [] | verdict pills: |
| banner (Ollama/mode) | [023] 2.10 | Y | ... | | correct Y/N |
| | [88] 0.90 | N | ... | | supported Y/N |
| [BM25 rank list with scores, truncation badge; | complete Y/N | halluc rate |
| token budget bar | | + per-case metrics (precision/recall/f1, tokens, laten
+-----+
```

GUI controls validate like ch1 §5.2 (non-empty question; $k \in [1, 100]$, $budget \geq 1$, $tiers \geq 1$ selected; `Cancel` enables only while running). On `Run` the pipeline executes off-thread (E-16); the panel shows the ranked BM25 list **with scores**, a truncation badge when `Context.truncated`, the grounded answer with its cited `sources`, and the judge verdict pills. **One question at a time** (R-13) — no batch/multi-turn.

6. Invariants (must hold in every valid implementation)

ID	Invariant	Verified by
I-001	<code>retrieval_pr</code> obeys the §18 arithmetic exactly for the worked example (expected {D3,D17,D42}, retrieved [D3,D17,D88,D91] $\Rightarrow$ TP=2,FP=2,FN=1, $P = 0.50$, $R \approx 0.667$, $F1 \approx 0.571$.),	T-05
I-002	Determinism (the §12 thesis): for fixed corpus + query + ($k, b, tokenizer$), <code>BM25Retriever.search</code> yields a byte-identical <code>ScoredDoc</code> list across runs; <code>build_context</code> and <code>aggregate</code> are pure functions of their inputs. <code>--mock</code> runs are bitwise-reproducible (R-15).	T-04, T-07
I-003	Grounding / anti-hallucination: every id in <code>Answer.sources</code> $\subseteq$ <code>Context.provenance</code> , the harness rejects/flags any answer that cites a doc outside the retrieved context. The judge’s <code>supported=True</code> is only acceptable when every claim is traceable to a provided doc.	T-09
I-004	<code>Context.tokens</code> $\leq$ <code>token_budget</code> always; <code>Context.truncated == True</code> iff at least one doc was dropped to fit (E-05/I-006).	T-06
I-005	<code>est_tokens(s)</code> is a single deterministic formula used identically by the context builder and the report (the reported <code>context_tokens</code> equals what was built, I-006).	T-06

ID	Invariant	Verified by
I-006	Reported <code>context_tokens</code> and <code>truncated</code> exactly mirror the <code>Context</code> actually assembled for that case (<code>report</code> $\equiv$ <code>build</code>).	T-11
I-007	No division by zero. <code>Precision=None</code> when <code>TP+FP==0</code> ; <code>Recall=None</code> when <code>TP+FN==0</code> ; <code>F1=0.0</code> when <code>P+R==0</code> ; <code>hallucination_rate=0.0</code> when <code>sum(total_factual_claims)==0</code> . A row with no retrieval output cannot contribute to a mean.	T-05, T-08
I-008	Failure attribution (R-12): every terminal <code>ERROR</code> / <code>PARTIAL</code> case names exactly one <code>failure_stage</code> ; retrieval-stage fields are populated for any case that cleared <code>RETRIEVING</code> , so a generation/judge fault still yields a full retrieval diagnosis.	T-10
I-009	The LLM is used in exactly two places — <code>LLM.generate</code> and <code>Judge</code> (R-05 / R-06). <code>retrieval.py</code> , <code>context.py</code> , <code>metrics.py</code> , <code>corpus.py</code> contain no LLM/Ollama reference (source-scan; ch1 T-02 I-002 analog).	T-02
I-010	No syntactically-valid-but-out-of-schema object is accepted: a panel/row reaches <code>COMPLETED</code> / <code>JUDGED</code> only via a <code>jsonschema</code> -valid object (ch1 I-009). An out-of-range <code>confidence</code> or missing <code>required</code> field $\Rightarrow$ reject/retry/ <code>ERROR</code> , never <code>COMPLETED</code> .	T-08
I-011	No Ollama/network is required to import the package or run the test suite ; the suite drives <code>MockLLM</code> + <code>MockJudge</code> only (R-14).	T-02 , T-14
I-012	<code>aggregate.by_tier</code> contains one sub-aggregate per populated tier; the root <code>aggregate</code> equals the cross-tier combination of the same formulas.	T-08
I-013	A case whose <code>relevant_docs</code> reference an id absent from the corpus is a load-time error , not a silent 0-recall (fail fast, E-17).	T-15
I-014	The GUI's <code>Cancel</code> /error path leaves no live worker and a terminal panel (ch1 I-010 / E-08 analog; E-16).	T-16 (offscreen)

7. Constraints (precise and measurable)

ID	Constraint	Measurement
K-01	The test suite runs fully offline (no Ollama, no network, no model download) in < 90s on a dev box; it never imports or contacts the Ollama daemon.	T-14
K-02	Retrieval + context + metrics (the deterministic boundary) run for the entire 100-doc / 25-question default dataset in < 5s with the <code>MockLLM</code> double.	smoke / T-13
K-03	Default parameters: <code>k=5</code> , <code>token_budget=2000</code> , <code>k1=1.5</code> , <code>b=0.75</code> , <code>max_retries=2</code> , <code>timeout_s=60</code> , <code>seed=42</code> . All overridable via CLI flags (§5.1).	T-13
K-04	The deterministic pipeline (retrieve + context + metrics) is network- and LLM-free — importable and runnable with zero external services (R-17 , I-009).	T-02
K-05	A single real end-to-end <code>--model qwen3.8:27b-mlx</code> eval of the full default dataset may take minutes (27B local inference + per-question judging); it is opt-in / manual only , never on the automated path.	§9.5 smoke

8. Edge cases and failure semantics

ID	Situation	Required behavior
E-01	Malformed corpus entry / unreadable <code>documents/NNN.txt</code>	<code>load_corpus</code> raises at load time with the offending path; the CLI exits 3 (§5.1). Never silently index a partial corpus.
E-02	k-th retrieval returns 0 matches (empty score, or k exceeds non-empty score count)	<code>search</code> returns <code>[]</code> (or the available matches); context is empty; the harness records <code>retrieved=[]</code> , <code>tp=0</code> , <code>fn=len(expected)</code> , <code>recall = 0/(0+FN) = 0.0</code> , <code>precision = None</code> (I-007), and still produces an ERROR/PARTIAL row with <code>failure_stage="retrieval"</code> when the answer cannot be grounded — no crash (§3 of ch2: empty context is a <i>result</i> , not a fault).
E-03	FP-only retrieval (retrieved docs are all irrelevant to <code>relevant_docs</code>)	<code>TP=0</code> , <code>FP=k</code> , <code>precision = 0.0</code> , <code>recall = 0.0</code> , <code>f1 = 0.0</code> . This is the <i>context pollution</i> case (§6/§7) — the report must not crash on <code>FP>0</code> .
E-04	FP-only + FN-only combined (the §18 example: $P+R \neq 0$ here, but guard the $P+R=0$ degenerate case)	<code>f1 = 0.0</code> explicitly (not <code>inf/nan</code>) (I-007).
E-05	<code>token_budget</code> too small to fit even the top document	Include the partial text of as many highest-ranked docs as fit (or 0 if none fits); <code>truncated=True</code> ; <code>Context.tokens <= budget</code> still holds (I-006). The prompt reflects whatever actually fit.
E-06	Duplicated documents (identical text, different ids)	<code>build_context(dedupe=True)</code> keeps the highest-score instance (by <code>rank</code> ; ties by <code>doc_id</code>), drops the rest; the kept id is the one in <code>provenance/sources</code> . (Dedupe is part of §3’s “deduplicate” step in the C-03 builder.)
E-07	Answer LLM emits non-JSON or an out-of-schema object	<code>ch1-style</code> : strip optional <code>```json</code> fence, retry up to <code>max_retries</code> with the last error appended to the system prompt; on exhaustion <code>Answer(status="ERROR")</code> , and the case <code>failure_stage="generation"</code> (I-008). Never an unvalidated dict downstream.
E-08	Answer <code>sources</code> reference a doc id not in the retrieved context (model “cites” something it wasn’t given)	Deterministically drop the foreign ids from <code>Answer.sources</code> and flag <code>grounding_violation</code> on the row; if any were dropped, <code>supported</code> is forced <code>False</code> and the claim is counted. Enforced in the harness (I-003), not trusted to the model.
E-09	Distractor tier : a relevant fact shares terminology with an irrelevant/contradictory policy	BM25 returns the distractor(s) alongside true support; the case is <i>measured</i> , not special-cased — the distractor manifests as reduced <code>precision</code> and/or a <code>supported=False</code> /unsupported claim. This is the <i>interesting</i> regime (§17).
E-10	Judge LLM emits non-JSON / out-of-schema verdict / fails	Retry like E-07; on exhaustion <code>Verdict(status="ERROR")</code> , case $\rightarrow$ PARTIAL with retrieval metrics intact and <code>correct/supported/complete=None</code> (R-12: retrieval is still diagnosable). <code>--judge off</code> skips this entire stage (retrieval-only eval, E-12).

ID	Situation	Required behavior
E-11	Ollama daemon unreachable (/api/tags or /api/chat connection error)	If <code>--mock</code> or no <code>--model</code> forced: fall back to MockLLM+MockJudge and print a banner <code>Ollama unavailable - using mock pipeline</code> (ch1 E-13 analog). If an explicit real <code>--model</code> was requested <i>and</i> Ollama is down: CLI exits 4 with <code>ollama not reachable</code> ; re-run with <code>--mock</code> or start ollama. Never a hang.
E-12	<code>--model NAME</code> names a model not pulled locally (Ollama 404 / model not found, ch1 E-14)	That run errors with <code>model not found: 'NAME'</code> - pull it with: <code>ollama pull NAME</code> ; with <code>--mock</code> the run proceeds on the mock instead. <code>--judge off</code> makes <i>generation</i> still required but skips the judge stage.
E-13	<code>--seed N</code> requested	MockLLM/corpus+questions generation honor the seed for reproducibility (R-15); the real Ollama path passes <code>options.seed=N, temperature=0.0</code> (best-effort; bitwise text not asserted, ch1 R-15).
E-14	GUI Cancel mid-generation	Worker is torn down; panel → terminal <code>ERROR</code> with <code>failure_stage="generation"</code> (a user cancel); no live worker survives; siblings (n/a — single-question GUI) unaffected (I-014 , ch1 E-08/I-010).
E-15	A <code>questions.json</code> <code>relevant_docs</code> entry names an id absent from the corpus	Load-time error (fail fast): <code>load_questions(corpus)</code> raises; CLI exits 3 (§5.1). Ground truth integrity is a precondition of any precision/recall measurement — a silent 0/None would corrupt the baseline (I-013).
E-16	GUI Run while a previous run is active	Cancel the prior worker first; exactly one active pipeline at a time (I-014 , ch1 E-12).
E-17	Empty question / dataset (question blank, or <code>--tiers</code> yields zero matching questions)	Blank question → CLI errors <code>question must be non-empty</code> ; <code>--tiers</code> matching zero → warning + exit 0 with an empty report (no crash, no division by zero — I-007).

Failure philosophy: the LLM (generate and judge) is the *probabilistic* side of the ch1 §15 reliability boundary; everything else — retrieval, context, metrics — is the **deterministic** side and is the *source of truth* for the diagnosis ([I-008/R-12](#)). The dominant failure mode here is **attributing a retrieval failure to a generation failure** (or vice-versa) — the single thing this lab exists to make *observable* (§18). `--judge off` implements the “did retrieval fail, or did the model fail to use it?” ablation: with judging off, the report still carries full precision/recall per case ([E-10/R-12](#)), proving the retrieval half independently. MockLLM + MockJudge are the deterministic doubles that let the *entire* suite assert this without any model in the loop ([I-011/R-14](#)).

9. Acceptance criteria, tests, and evals

All tests target **Level-3 executable** criteria. The deterministic layers (retrieve, context, metrics, corpus, MockLLM, MockJudge) need **no Qt, no Ollama, no network**; the GUI path is offscreen; the only *real* model call is the **manual smoke** (§9.5).

9.1 Corpus and question dataset (§15, §17)

ID	Criterion
T-01	<code>gen-corpus --n-docs 100 --n-questions 25 --seed 42</code> writes exactly 100 distinct documents/ <code>NNN.txt</code> and a <code>questions.json</code> of 25 questions, each with <code>question</code> , <code>gold_answer</code> , non-empty <code>relevant_docs</code> (<code>ids ⊆ corpus</code>), and a <code>tier</code> $\in$ the four §17 tiers; two invocations with the same seed produce byte-identical files (R-15).
T-01a	<code>load_corpus</code> + <code>load_questions</code> accept the generated artifacts and raise on a blank/missing <code>relevant_docs</code> or a <code>relevant_docs</code> id absent from the corpus (E-15, I-013).
T-01b	The 25-question set has a non-trivial per-tier distribution (each tier present, with a <code>distractor</code> question whose <code>relevant_docs</code> are <i>also</i> present as lexically-similar-but-irrelevant docs in the corpus).

9.2 Retrieval (deterministic — R-02, R-17, no LLM)

ID	Criterion
T-04	Determinism (I-002) : <code>BM25Retriever(corpus).search(q, k)</code> is byte-identical across two builds with the same corpus + params; the tie-break (<code>doc_id</code> asc, then score desc) is respected by a crafted duplicate-score corpus.
T-05a	The §18 worked example (I-001) : <code>retrieval_pr(expected={D3,D17,D42}, retrieved=[D3,D17,D88,D91]) $\Rightarrow TP = 2, FP = 2, FN = 1, Precision = 0.50, Recall \approx 0.667, F1 \approx 0.571$</code> .
T-05b	Guards (I-007) : empty <code>retrieved</code> $\Rightarrow$ <code>precision=None</code> ; all-irrelevant $\Rightarrow$ <code>precision=0.0</code> ; <code>P+R==0</code> $\Rightarrow$ <code>f1=0.0</code> (no inf/nan).
T-05c	Ranking sanity : on a seeded corpus a question's <code>relevant_docs</code> appear in the top <code>k</code> of their own search (<code>recall ≈ 1</code>) for the <i>easy/multi</i> tiers — the <i>hard</i> and <i>distractor</i> tiers are <i>allowed</i> to miss, but never crash.

9.3 Context + budgets (deterministic — R-03, §14, §3)

ID	Criterion
T-06	Budget (I-004/I-006) : <code>build_context(scored, token_budget=N)</code> always yields <code>Context.tokens <= N</code> ; <code>truncated=True</code> iff a doc was dropped. A crafted <code>token_budget</code> smaller than the top doc forces <code>truncated=True</code> with a non-empty prompt (E-05).
T-06a	Dedupe (E-06/I-004) : a two-doc dedupe of identical text keeps the highest-rank instance and sets <code>truncated=True</code> .
T-06b	Provenance : every id in <code>Context.provenance</code> $\subseteq$ the included docs\“, <code>andest_tokens</code> is the same formula used by the builder and by the report (I-005).

9.4 LLM + Judge + Metrics (offline, MockLLM + MockJudge — R-05/06/07/08/09)

ID	Criterion
T-08	Schema gate (I-010) : the answer schema rejects an out-of-range confidence (e.g. 1.5) and a missing required field; a malformed verdict is likewise rejected; both retry up to <code>max_retries</code> then set <code>status="ERROR"</code> . Only jsonschema-valid objects reach COMPLETED/JUDGED.

ID	Criterion
T-08a	MockJudge determinism + hallucination math (R-09): for a crafted case whose MockLLM answer contains one claim absent from the context, MockJudge reports <code>unsupported_claims</code> and <code>total_factual_claims</code> such that <code>hallucination_rate = unsupported/total</code> ; MockJudge is bitwise-reproducible under a fixed seed (R-15). <code><hallucination rate with an all-supported answer = 0.0; with zero total claims = 0.0 ([I-007](#sec-invariants)).></code>
T-08b	Aggregate (I-012/R-08): over a synthetic row-set, <code>aggregate</code> yields the exact mean <code>precision/recall/f1/answer_accuracy</code> over the non-None rows only; <code>by_tier</code> has one sub-aggregate per tier present.
T-08c	Grounding gate (I-003/E-08): an <code>Answer</code> sourcing a doc id not in <code>Context.provenance</code> is forced <code>supported=False</code> and the foreign ids are stripped.

9.5 Manual / real-model smoke (opt-in — not in `uv run pytest`)

- `uv run rag-eval --gen-corpus --seed 42` generates the corpus + questions (< 1s, deterministic, T-01).
- `uv run rag-eval --mock` runs the **full pipeline offline** over the generated dataset, prints the §9.5 summary, and writes `report.json` (< 5s, K-02). A human reviews a **distractor-tier** case in `report.json` to confirm the precision drop is *observable* (§7 context pollution / §17 distractors).
- `uv run rag-eval --model qwen3.8:27b-mlx` runs the **real LLM+judge** over the full dataset (K-05, minutes on a 27B local model); the report is compared to the `--mock` run so the human sees *which* differences are retrieval-driven vs model-driven (the core diagnostic of §18). **This is the only automated path that talks to Ollama; it is never in `uv run pytest` (I-011).**
- **GUI smoke (offscreen + one real interactive run):** launch `uv run rag-gui`; confirm the BM25 ranking + scores + truncation badge render for a `--model` run *and* for the `--mock` run (ch1 §9.5 analog).

9.6 Structure / architecture (ch1 T-02 analog — I-009, R-17)

ID	Criterion
T-02	LLM-is-only-in-two-places (I-009): a source scan of <code>retrieval.py</code> , <code>context.py</code> , <code>metrics.py</code> , and <code>corpus.py</code> finds no reference to <code>OllamaClient</code> , <code>LLM</code> , <code>Judge</code> , <code>Ollama</code> , <code>httpx</code> , or any model name. (<code>httpx/OllamaClient</code> appear only in <code>model.py/judgment.py</code> .) The suite imports and runs with zero external services (I-011/R-14).
T-14	Offline full-suite (K-01, I-011): <code>uv run pytest</code> (default) passes with no Ollama daemon, no network, and no model — all cases drive MockLLM+MockJudge.
T-15	Ground-truth integrity (E-15, I-013): <code>load_questions</code> raises on a <code>relevant_docs</code> id absent from the loaded corpus; the CLI exits 3 in that case (asserted via a synthetic corrupt <code>questions.json</code>).
T-16	GUI offscreen (I-014, E-08/E-16): starting a GUI run while one is active cancels the prior; after <code>Cancel</code> zero workers are alive; the panel is in a terminal <code>ERROR</code> with <code>failure_stage="generation"</code> — offscreen with <code>QT_QPA_PLATFORM=offscreen</code> .

10. Dependencies and environment

Concern	Decision	Rationale
Package/env manager	uv	Fast, reproducible, pins Python; satisfies R-14 .
Python Retrieval	3.12 (≥ 3.12 , < 3.13) pure-Python BM25 (stdlib: <code>collections</code> , <code>math</code> , <code>re</code>)	Stable; consistent with <code>ch1</code> . R-02/R-17 — deterministic, no embedding model/network; K-01/K-04 . <i>No</i> <code>numpy</code> / <code>sklearn</code> / <code>transformers</code> (out of scope, Q-01).
Schema validation	jsonschema	C-04 answer + verdict schemas (<code>ch1 C-05</code>).
GUI (optional)	PyQt5 (5.15)	R-13 ; <code>ch1</code> analog. Not required for the CLI or the test suite.
HTTP (real LLM only)	httpx	Ollama transport in <code>model.py</code> / <code>judgment.py</code> only — never in the deterministic layers (I-009); <code>ch1 C-03b</code> analog.
Local inference engine	Ollama (<i>external</i>) <code>qwen3.8:27b-mlx</code> (5642e97495e1, ~18GB)	Generation + judging runtime at <code>http://localhost:11434</code> (§0). Not a Python dependency; degrades to <code>MockLLM</code> / <code>MockJudge</code> (E-11/E-12).
Mock doubles	in-repo <code>MockLLM</code> , <code>MockJudge</code>	Deterministic, offline, reproducible (R-14/R-15); drive the entire automated suite.
Corpus + questions	in-repo generator (<code>gen_corpus.py</code> , <code>gen_corpus_and_questions</code>)	Deterministic ground truth (§15); seeded (R-15); regenerable.
Dev deps	pytest , pytest-qt , ruff	Automated §9 suite + lint.
GUI test backend	<code>QT_QPA_PLATFORM=offscreen</code>	Headless CI (<code>ch1 E-10</code>).

Proposed layout (derived from §3–§5):

```

src/rag_eval/
  types.py      # C-01/C-04: Document, ScoreDoc, Context, Question, Answer, Verdict, Usage, RunMetrics
  corpus.py     # C-01 load_corpus / gen_corpus + gen_corpus_and_questions (deterministic ground truth,
  retrieval.py  # C-02 BM25Retriever (deterministic; 0-1 formula, 0-1b tie-break) [no LLM -
I-009]
  context.py    # C-03 build_context + est_tokens (token budget; dedupe/truncate/label) [no LLM -
I-009]
  metrics.py    # C-07 retrieval_pr + aggregate (P/R/F1, accuracy, hallucination, guards) [no LLM -
I-009]
  model.py      # C-05 LLM/ABC + OllamaLLM + MockLLM (+ OllamaClient via httpx) [probabilistic]
  judgment.py   # C-06 Judge/ABC + OllamaJudge + MockJudge [probabilistic]
  schemas.py    # the answer + verdict JSON-Schema objects (ch1 C-05 analog)
  pipeline.py   # C-pipeline: run_case(question) + run_dataset(dataset) → metrics
  cli.py        # §5.1 rag-eval (eval / gen-corpus / show)
  ui.py         # §5.2 rag-gui (optional PyQt5; reuses the above)
  app.py        # main() entry points
schemas/
  answer.json   verdict.json    # the two structured schemas
questions.json  documents/NNN.txt    # generated by `gen-corpus` (T-01)
tests/          # §9 (pure + offscreen GUI); conftest.py forces QT_QPA_PLATFORM=offscreen

```

Reproducibility (see future README.md):

```

# host prerequisite for the REAL path (optional; the mock path needs neither):
# ollama pull qwen3.8:27b-mlx      # already present on this box (ID 5642e97495e1, ~18GB)
uv sync                            # create .venv (Python 3.12), install everything
uv run rag-eval gen-corpus --seed 42  # generate the ~100-doc corpus + questions.json (T-01)
uv run pytest                      # run the §9 suite - FULLY OFFLINE, no Ollama needed (I-011, K-01)
uv run rag-eval --mock              # full pipeline offline → report.json (<5s, K-02)
uv run rag-eval --model qwen3.8:27b-mlx  # REAL LLM+judge over the dataset (opt-in; K-05, minutes)
uv run rag-gui                      # optional GUI over the same pipeline (Ollama if reachable, else mock)

```

11. Traceability matrix (id → where realized)

Spec id / requirement	Where realized (component / module)	Verified by (tests / evidence)
§15 core / R-01	pipeline.py (run_case/run_dataset)	T-01, §9.5
R-02 / I-002	C-02 BM25Retriever, O-1b tie-break	T-04
R-03 / I-004/006	C-03 build_context + est_tokens	T-06, T-06a
R-04 / I-003	pipeline.py grounding gate	T-08c, E-08
R-05 / I-010	C-05 LLM.generate, schemas.py/answer.json	T-08
R-06 / I-010	C-06 Judge + schemas/verdict.json	T-08, T-08a
R-07 / I-001/007	C-07 retrieval_pr, §18 worked example	T-05a, T-05b
R-08	C-07 aggregate (answer_accuracy)	T-08b
R-09 / I-007	C-07 aggregate (hallucination_rate)	T-08a, T-08b
R-10	gen_corpus_and_questions (4 tiers)	T-01b, E-09
R-11	cli.py eval (report + per-tier)	§9.5
R-12 / I-008	RunMetrics.failure_stage + -judge off	T-10, E-10
R-13	ui.py (offscreen, ch1 analog)	T-16
R-14 / I-011	MockLLM+MockJudge + pyproject	T-14
R-15 / I-002	seed threading (corpus + mock paths)	T-01, T-07, T-08a
R-16	model.py list_models + fallback banner	E-11, E-12
R-17 / I-009	no-LLM layers scan	T-02
I-013 / E-15	load_questions integrity check	T-15
§3 context resource	est_tokens O-2 + token budget	T-06, K-03
§6/§7 pollution	distractor tier + E-03/E-09	T-01b, §9.5 smoke
§18 “retrieve vs reason”	-judge off ablation + failure_stage	R-12, T-10
K-01 / I-011	offline full suite (no Ollama)	T-14
K-04 / I-009	deterministic boundary network-free	T-02
§12	BM25+context+metrics pure	I-002, T-04, T-06
deterministic-pipeline		

Open questions / ambiguities flagged for the human (spec elicitation):

1. **Retrieval mechanism.** Spec picks **deterministic BM25** (your answer). Dense/nomic-embed-text/hybrid are out-of-scope extensions — *confirm* this is acceptable for v0.1 (Q-01). If a real vector index is later wanted, the BM25Retriever search interface (ranked ScoreDoc) is the seam; a DenseRetriever would slot in behind the same interface.
2. **Answer + verdict schemas.** Both are fixed JSON-Schema objects for v0.1 (ch1 Q-01 analog):
answer {answer, confidence, sources}; verdict {correct, supported, complete, unsupported_claims, total_factual_claims, rationale}. *Confirm* these shapes (Q-02).
sources may be empty when the answer is "I cannot answer from the provided documents." — *confirm* that empty sources is allowed (the minItems:0 default).

3. **Judge model.** `Judge.model_id` defaults to the **same** `qwen3.8:27b-mlx` as generation (your answer); it may name a different model. *Confirm* single-model-for-both is fine, or whether a cheaper judge model should be the default (Q-03).
4. **Token estimator.** `est_tokens = ceil(len/4)` (approx; deterministic, I-005) — *confirm* this vs. a real tokenizer (out of scope for v0.1: it would introduce a model dependency into the deterministic boundary, contradicting I-009; Q-04).
5. **Concurrency.** v0.1 is **strictly sequential** per case (one shared `qwen3.8:27b-mlx` slot; deterministic and simplest). `--concurrency N` is deferred (Q-05).
6. **State/memory extension.** §10/§11 (state $\neq$ context; memory) are explicitly **out of scope** for v0.1 (Q-06): the corpus + question set are static per run. *Confirm* no persistence between questions is needed.
7. **Ground truth in a synthetic corpus.** The `gen-corpus` generator authors the `question` $\leftrightarrow$ `relevant_documents` mapping deterministically so precision/recall have a meaningful denominator. *Confirm* this is acceptable as the eval baseline for v0.1 (Q-07) vs. hand-authoring a curated 10-doc + 5-question starter set first to sanity-check the pipeline before scaling to 100.
8. **Report format.** v0.1 emits JSON (`report.json`) + a human summary to stdout (Q-08). *Confirm* no richer format (HTML/CSV dashboard) is needed for v0.1.

End of specification. This document is the source of truth; implementation and tests are to be derived from it and kept in sync per §11. v0.1 covers §15–§21 of `curriculum/week1/chapter2.md`.